

Patrones en Arquitectura de Microservicios

Patrones mínimos requeridos para una arquitectura robusta

¿Cuántos patrones se requieren como mínimo?

5-7

patrones

¿Por qué importante?

En microservicios, los patrones son importantes porque ayudan a resolver problemas comunes de comunicación, resiliencia, despliegue y observabilidad en sistemas distribuidos.

Recomendación mínima

Como mínimo, suele recomendarse un conjunto de 5 a 7 patrones esenciales para cubrir las necesidades básicas de arquitectura y operación.

Beneficio principal

Estos patrones son clave porque permiten mantener el sistema más estable, escalable y fácil de evolucionar.

Patrones Esenciales de Microservicios



API Gateway

Nivel: Microservicios

Por qué: Centraliza todo el acceso



Event Driven / Observer

Nivel: Microservicios +
Comportamiento

Por qué: Comunicación
desacoplada

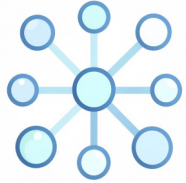


Circuit Breaker

Nivel: Microservicios

Por qué: Evita caídas del sistema

Patrones Mínimos Requeridos



API Gateway

MICROSERVICES

Centraliza todo el acceso.



Event Driven / Observer

MICROSERVICES +
COMPORTAMIENTO

Comunicación desacoplada.



Circuit Breaker

MICROSERVICES

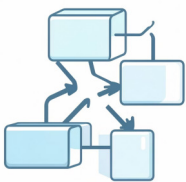
Evita caídas del sistema.



Factory Method

CREACIONAL

Flexibilidad y bajo
acoplamiento.

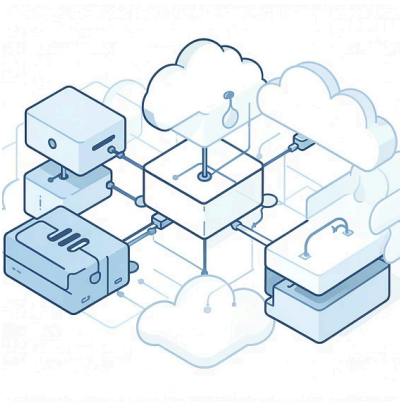


Builder

CREACIONAL

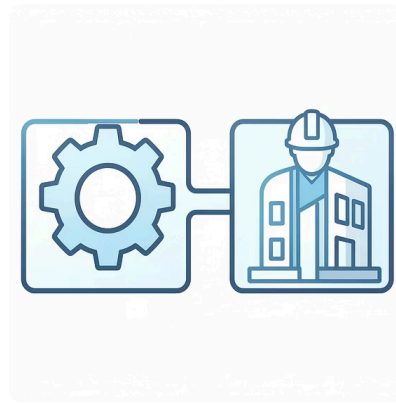
Construcción flexible de
objetos.

Categorías de Patrones en Microservicios



Patrones de Microservicios

- API Gateway
- Event Driven / Observer
- Circuit Breaker



Patrones Creacionales

- Factory Method
- Builder



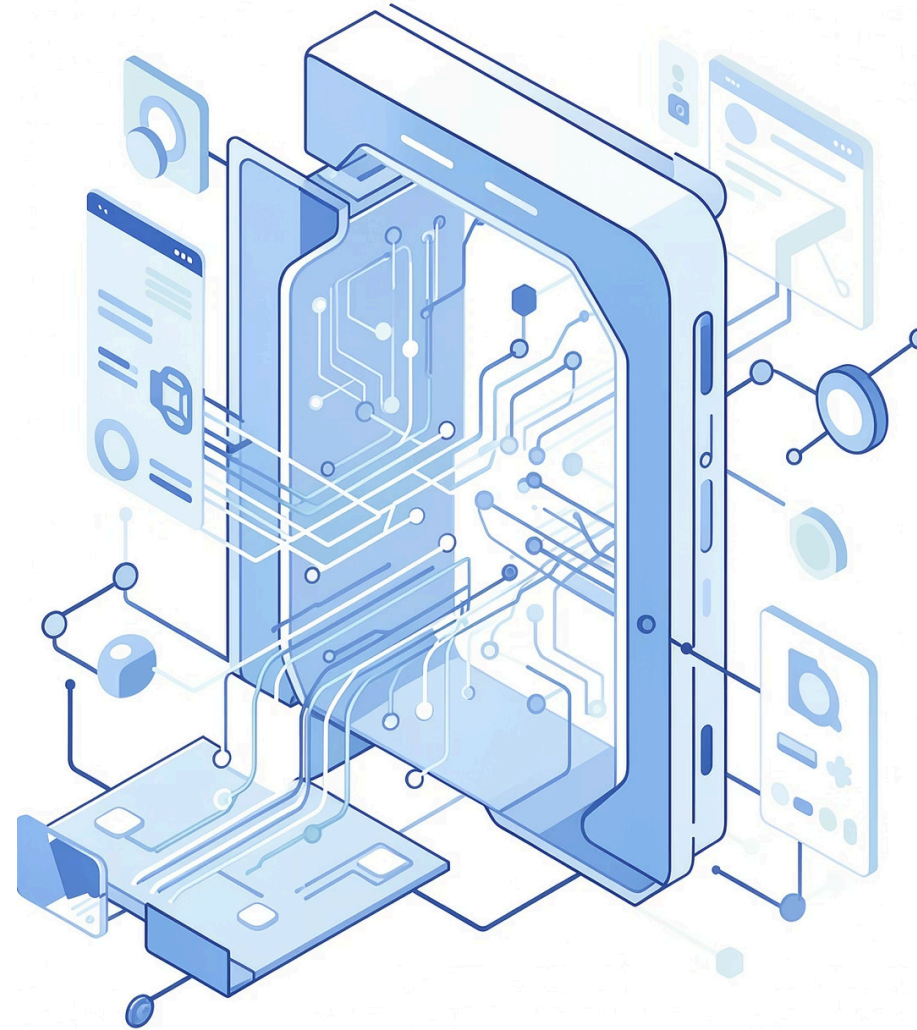
Complementarios (Recomendados)

- SOLID (Principios de diseño)
- CLEAN CODE (Calidad de código)
- C4 (Modelado de arquitectura)

API Gateway - ¿Qué es?

El API Gateway es una puerta principal que recibe todas las solicitudes de los clientes y las enruta al microservicio correcto.

- El cliente no accede directamente a cada servicio
- Todo pasa primero por el Gateway
- Actúa como intermediario centralizado



API Gateway - ¿Por qué es importante?



Organiza las peticiones de forma centralizada



Mejora la seguridad del sistema



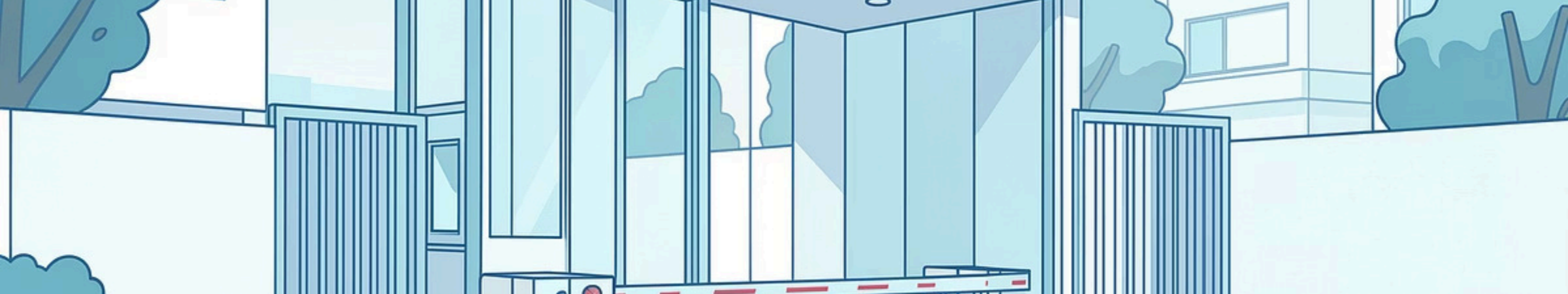
Evita conexiones desordenadas



Centraliza el acceso y control



Facilita autenticación y autorización



API Gateway - Ejemplo cotidiano

Una portería de un conjunto residencial

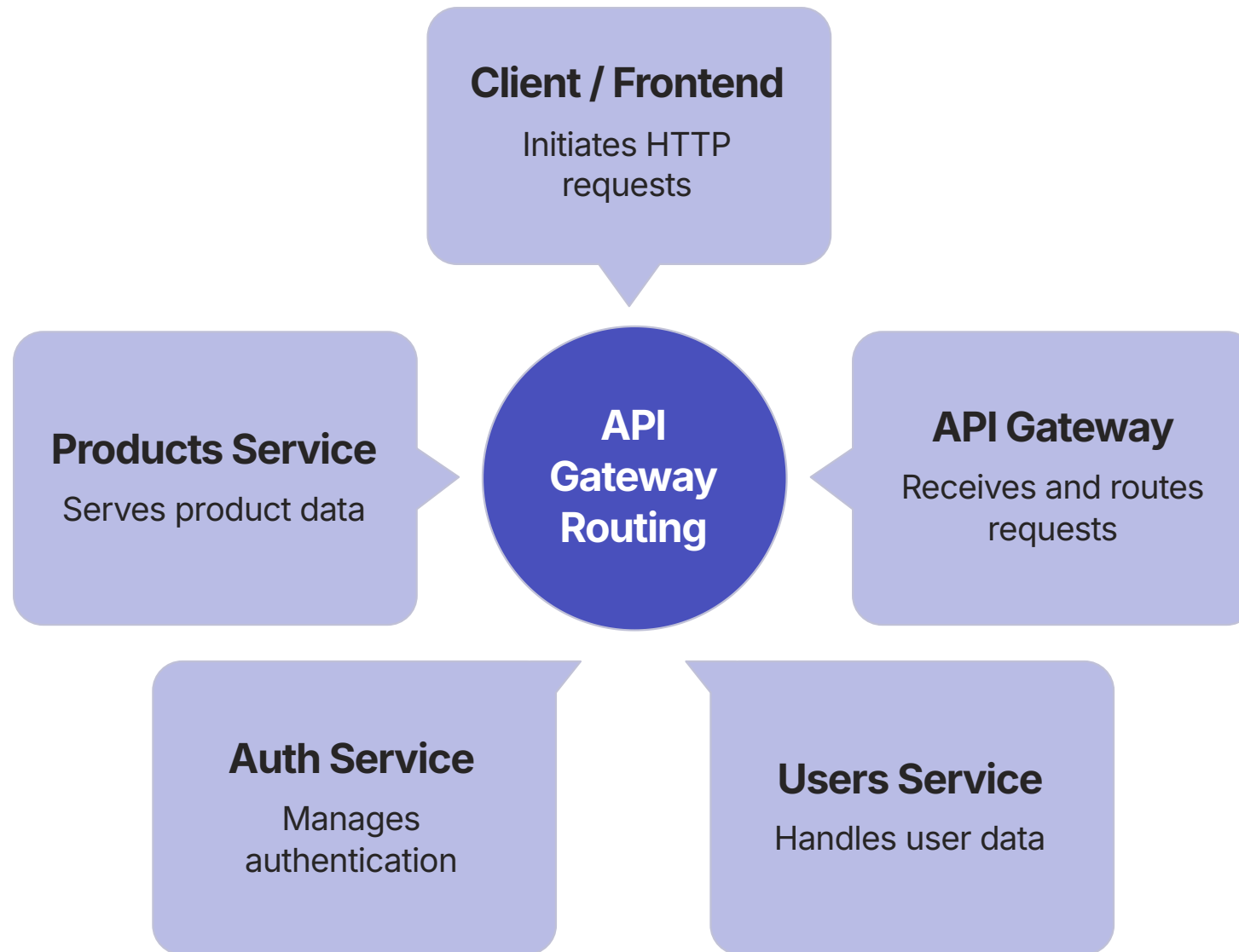
Cuando una persona llega a un conjunto:

- La portería pregunta: "¿A dónde va?"
- Verifica la identidad del visitante
- Luego lo dirige a la torre correcta
- La persona NO entra directamente a cualquier apartamento

Relación:

- Portería = API Gateway
- Visitante = Cliente/Usuario
- Torres = Microservicios
- Verificación = Autenticación

API Gateway - En microservicios



El API Gateway recibe todas las solicitudes del cliente y las distribuye a los microservicios correspondientes según la ruta solicitada.

API Gateway - Ejemplo sencillo en código

routes:

- /users -> puerto 8081 (Users Service)
- /auth -> puerto 8082 (Auth Service)
- /products -> puerto 8083 (Products Service)

¿Qué significa?

- Si un cliente hace una solicitud a: **/users** → el Gateway lo envía al Users Service
- **/auth** → el Gateway lo envía al Auth Service
- **/products** → el Gateway lo envía al Products Service

Event Driven / Observer - ¿Qué es?

El patrón Event Driven funciona mediante eventos. Cuando algo ocurre en un servicio, se envía una notificación y otros servicios reaccionan automáticamente.

El patrón Observer permite que varios servicios estén "escuchando" esos eventos sin estar directamente conectados.

- Un servicio genera un evento
- Otros servicios se suscriben a ese evento
- Cuando ocurre el evento, todos los suscritos reaccionan automáticamente
- No hay acoplamiento directo entre servicios

Event Driven / Observer - ¿Por qué es importante?



Comunicación automática entre servicios



Bajo acoplamiento entre componentes



Independencia entre servicios



Mejor escalabilidad del sistema



Facilita la reactividad y respuesta rápida



Event Driven / Observer - Ejemplo cotidiano

Un grupo de WhatsApp

Cuando alguien manda un mensaje en un grupo:

- Una persona escribe: "Hay reunión mañana"
- El mensaje se envía al grupo
- Todos los miembros reciben la notificación automáticamente
- Cada persona puede reaccionar de forma independiente

Relación:

- Grupo WhatsApp = Sistema de eventos
- Mensaje = Evento
- Personas del grupo = Microservicios suscritos
- Notificación = Reacción automática

Event Driven / Observer - En microservicios



Cuando un usuario se registra, se genera un evento que es capturado por múltiples servicios que reaccionan automáticamente sin estar directamente conectados.

Event Driven / Observer - Ejemplo sencillo en código

Crear evento:

```
class UserRegisteredEvent {  
    String nombre;  
    String email;  
}
```

Publicar evento:

```
eventBus.publish(new UserRegisteredEvent("Juan", "juan@email.com"));
```

Servicios reaccionando:

```
// Email Service escucha el evento  
@EventListener  
void onUserRegistered(UserRegisteredEvent event) {  
    emailService.sendWelcomeEmail(event.email);  
}  
  
// Audit Service escucha el evento  
@EventListener  
void onUserRegistered(UserRegisteredEvent event) {  
    auditService.logEvent("Usuario registrado: " + event.nombre);  
}
```

¿Qué significa?

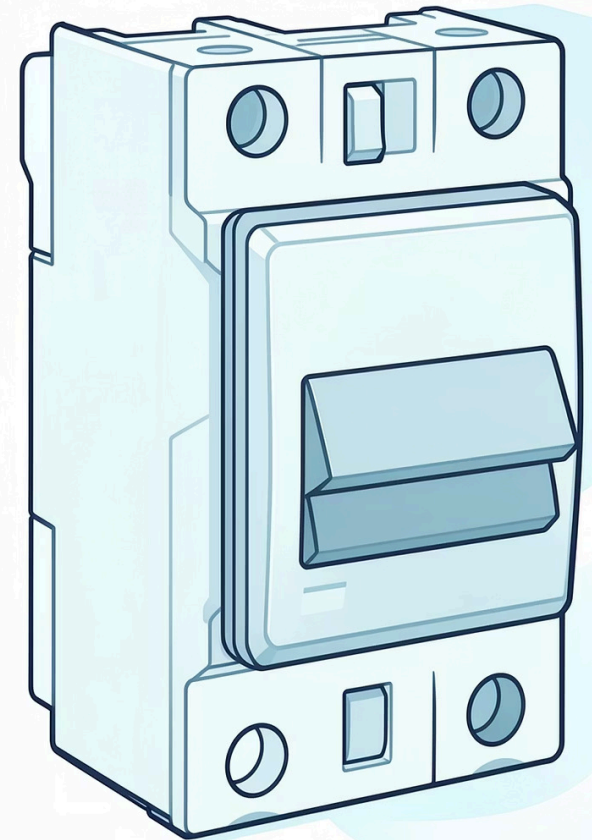
Cuando se publica el evento UserRegisteredEvent, todos los servicios suscritos reaccionan automáticamente sin estar directamente conectados.

Circuit Breaker - ¿Qué es?

El Circuit Breaker es un patrón que previene que un servicio intente comunicarse con otro servicio que está caído o no responde.

Funciona como un interruptor eléctrico:

- Cuando todo funciona bien: **CERRADO** (permite el paso)
- Cuando detecta fallos: **ABIERTO** (bloquea las solicitudes)
- Después de un tiempo: **SEMI-ABIERTO** (intenta recuperarse)
- Evita que los fallos se propaguen
- Protege el sistema de caídas en cascada
- Permite que los servicios se recuperen
- Mejora la resiliencia del sistema



Circuit Breaker - ¿Por qué es importante?



Evita caídas en cascada del sistema



Protege los servicios de sobrecarga



Mejora la resiliencia y tolerancia a fallos



Permite recuperación automática



Reduce el impacto de fallos en otros servicios



Circuit Breaker - Ejemplo cotidiano

Un interruptor eléctrico en casa

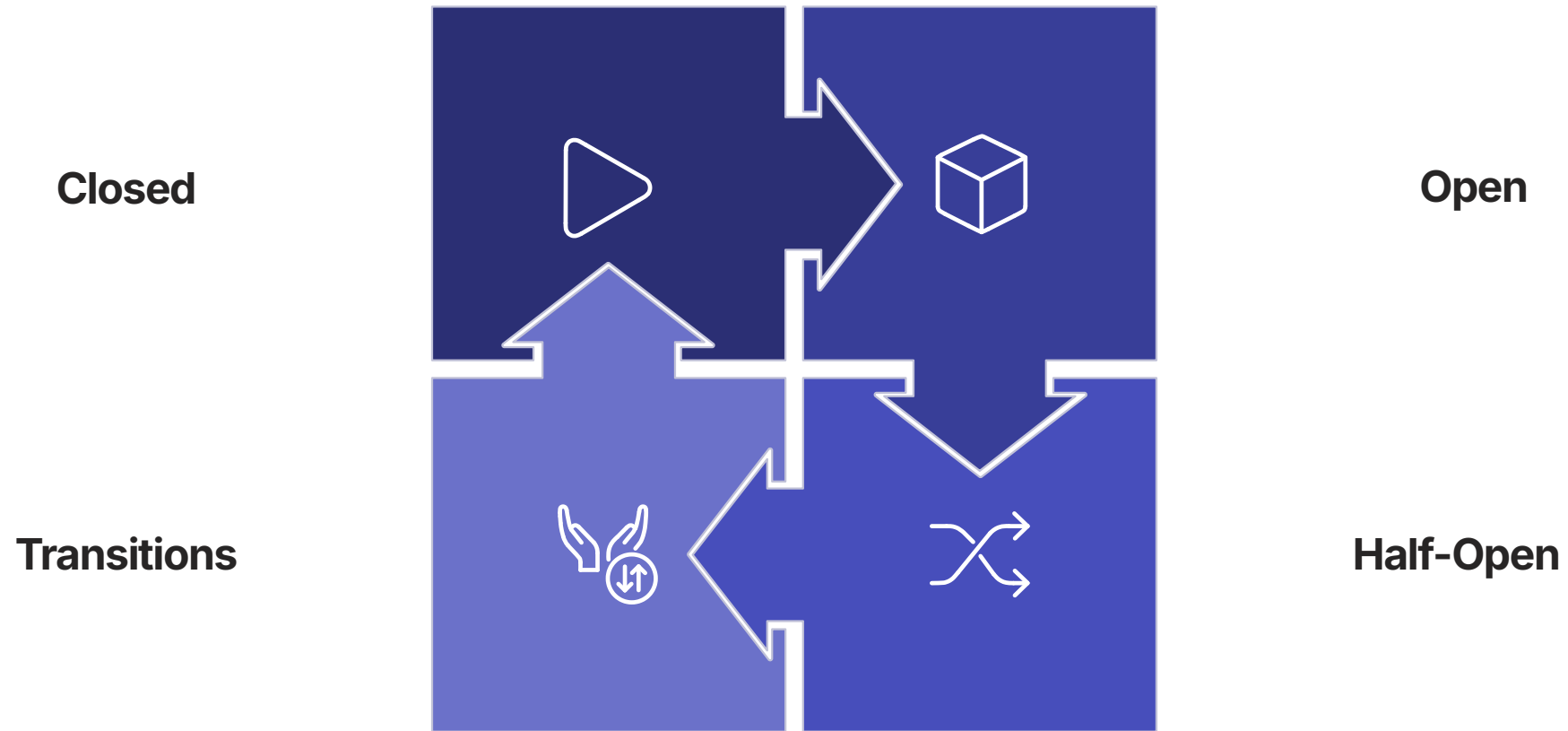
Cuando hay un cortocircuito en casa:

- El interruptor detecta el problema
- Automáticamente se abre (corta la electricidad)
- Protege los aparatos de daño
- Después de un tiempo, puedes intentar cerrar el interruptor
- Si el problema persiste, se abre de nuevo

Relación:

- Interruptor = Circuit Breaker
- Cortocircuito = Servicio caído
- Aparatos = Otros servicios
- Corte de electricidad = Bloqueo de solicitudes

Circuit Breaker - En microservicios



El Circuit Breaker monitorea las solicitudes y cambia de estado automáticamente para proteger el sistema.

Circuit Breaker - Ejemplo sencillo en código

Configurar Circuit Breaker:

```
@CircuitBreaker(  
    failureThreshold = 5,    // Abre después de 5 fallos  
    delay = 10000           // Intenta recuperarse después de 10 segundos  
)  
public Response callExternalService() {  
    return externalService.getData();  
}
```

Estados del Circuit Breaker:

CLOSED → Solicitud **normal**
OPEN → Rechaza solicitudes inmediatamente
HALF_OPEN → Intenta una solicitud de prueba

¿Qué significa?

- Si hay 5 fallos consecutivos, el circuito se abre
- Las nuevas solicitudes son rechazadas inmediatamente
- Después de 10 segundos, intenta una solicitud de prueba
- Si funciona, vuelve a CLOSED; si falla, vuelve a OPEN